Document number: P0022
Date: 2015-06-30
Project: Programming Language C++, Library Working Group
Reply-to: Eric Niebler <eniebler@boost.org>,

Proxy Iterators for the Ranges Extensions

> “Never do that by proxy which you can do yourself.”
>
> – *Italian proverb*

1 Introduction

This paper presents an extension to the Ranges design[4] that makes *proxy iterators* full-fledged members of the STL iterator hierarchy. This solves the “`vector<bool>`-is-not-a-container” problem along with several other problems that become apparent when working with range adaptors. It achieves this without fracturing the `Iterator` concept hierarchy[1] and without breaking iterators apart into separate traversal and access pieces[3].

The design presented makes only local changes to some Iterator concepts, fixes some existing library issues, and fills in a gap left by the addition of move semantics. To wit: the functions `std::swap` and `std::iter_swap` have been part of C++ since C++98. With C++11, the language got move semantics and `std::move`, but not `std::iter_move`. This arguably is an oversight. By adding it and making both `iter_swap` and `iter_move` customization points, iterators can control how elements are swapped and moved by permuting algorithms.

Also, there are outstanding issues in `common_type`, and by ignoring top-level cv- and ref-qualifiers, the trait is not as general as it could be. By fixing the issues and adding a new trait – `common_reference` – that respects cv- and ref-qualifiers, we kill two birds. With the new `common_reference` trait and `iter_move` customization point, we can generalize the Iterator concepts – `Readable`, `IndirectlyMovable`, and `IndirectCallable` in particular – in ways that bring proxy iterators into the fold.

Individually, these are simple changes the committee might want to make anyway. Together, they make a whole new class of data structures usable with the standard algorithms.

The design is presented as a series of diffs to the latest draft of the Ranges Extensions.

2 Implementation Experience

Everything suggested here has been implemented in C++11, where Concepts Lite has been simulated with the help of generalized SFINAE for expressions. In addition, a partial implementation using Concepts exists[2] that works with the "`-std=c++1z`" support in gcc trunk.

3 Motivation and Scope

The proxy iterator problem has been known since at least 1999 when Herb Sutter wrote his article "When is a container not a container?"[9] about the problems with `vector<bool>`. Because `vector<bool>` stores the `bool`s as bits in packed integers rather than as actual `bool`s, its iterators cannot return a real `bool&` when they are dereferenced; rather, they must return proxy objects that merely behave like `bool&`. That would be fine except that:

1. According to the iterator requirements tables, every iterator category stronger than InputIterator is required to return a real reference from its dereference operator (`vector` is required to have random-access iterators), and
2. Algorithms that move and swap elements often do not work with proxy references.

Looking forward to a constrained version of the STL, there is one additional problem: the algorithm constraints must accommodate iterators with proxy reference types. This is particularly vexing for the higher-order algorithms that accept functions that are callable with objects of the iterator's value type.

Why is this an interesting problem to solve? Any data structure whose elements are "virtual" – that don't physically live in memory – requires proxies to make the data structure readable and (optionally) writable. In addition to `vector<bool>` and `bitset` (which currently lacks iterators for no other technical reason), other examples include:

- A `zip` view of *N* sequences (described below)
- A view of elements in a database
- A view of elements in a different address space (e.g., in a different process or across the network)
- A view that does pre- and/or post-processing whenever an element is read or written (e.g., for logging purposes).
- A view of sub-objects (real or computed) that can only be accessed via getters and setters.

These are all potentially interesting views that, as of today, can only be represented as Input sequences. That severely limits the number of algorithms that can operate on them. The design suggested by this paper would make all of these valid sequences even for random access.

Note that not all iterators that return rvalues are proxy iterators. If the rvalue does not stand in for another object, it is not a proxy. The Palo Alto report[8] lifts the onerous requirement that Forward iterators have true reference types, so it solves the "rvalue iterator" problem. However, as we show below, that is not enough to solve the "proxy iterator" problem.

3.1 Proxy Iterator problems

For all its problems, `vector<bool>` works surprisingly well in practice, despite the fact that fairly trivial code such as below is not portable.

```
std::vector<bool> v{true,false,true};
auto i = v.begin();
bool b = false;
using std::swap;
swap(*i, b);      // Not guaranteed to work.
```

Because of the fact that this code is underspecified, it is impossible to say with certainty which algorithms work with `vector<bool>`. That fact that many do is due largely to the efforts of implementors and to the fact that `bool` is a trivial, copyable type that hides many of the nastier problems with proxy references. For more interesting proxy reference types, the problems are impossible to hide.

A more interesting proxy reference type is that of a `zip` range view from the range-v3[6] library. The `zip` view adapts two underlying sequences by building pairs of elements on the fly as the `zip` view is iterated.

```
vector<int> vi {1,2,3};
vector<string> vs {"a","b","c"};

auto zip = ranges::view::zip(vi, vs);
auto x = *zip.begin();
```

```
static_assert(is_same<decltype(x), pair<int&,string&>>{}, "");
assert(&x.first == &vi[0]);
assert(&x.second == &vs[0]);
```

The `zip` view's iterator's reference type is a prvalue `pair` object, but the `pair` holds lvalue references to the elements of the underlying sequences. This proxy reference type exposes more of the fundamental problems with proxies than does `vector<bool>`, so it will be used in the following discussion.

3.1.1 Permutable proxy iterators

Many algorithms such as `partition` and `sort` must permute elements. The Palo Alto report[8] uses a `Permutable` concept to group the constraints of these algorithms. `Permutable` is expressed in terms of an `IndirectlyMovable` concept, which is described as follows:

> The `IndirectlyMovable` and `IndirectlyCopyable` concepts describe copy and move relationships between the values of an input iterator, `I`, and an output iterator `Out`. For an output iterator `out` and an input iterator `in`, their syntactic requirements expand to:
>
> - `IndirectlyMovable` requires `*out = move(*in)`

The iterators into a non-const `vector` are `Permutable`. If we `zip` the two `vectors` together, is the resulting `zip` iterator also `Permutable`? The answer is: maybe, but not with the desired semantics. Given the `zip` view defined above, consider the following code:

```
auto i = zip.begin();
auto j = next(i);
*i = move(*j);
```

Since `*j` returns a prvalue `pair`, the `move` has no effect. The assignment then copies elements instead of moving them. Had one of the underlying sequences been of a move-only type like `unique_ptr`, the code would fail to compile.

The fundamental problem is that with proxies, the expression `move(*j)` is moving the *proxy*, not the element(s) being proxied. Patching this up in the current system would involve returning some special pair-like type from `*j` and overloading `move` for it such that it returns a different pair-like type that stores rvalue references. However, `move` is not a customization point, so the algorithms will not use it. Making `move` a customization point is one possible fix, but the effects on user code such a change are unknown (and unknowable).

3.1.2 Iterator associated types

The value and reference associated types must be related to each other in a way that can be relied upon by the algorithms. The Palo Alto report defines a `Readable` concept that expresses this relationship as follows (updated for the new Concepts Lite syntax):

```
template< class I >
concept bool Readable =
    Semiregular<I> && requires (I i) {
        typename ValueType<I>;
        { *i } -> const ValueType<I>&;
    };
```

The result of the dereference operation must be convertible to a const reference of the iterator's value type. This works trivially for all iterators whose reference type is an lvalue reference, and it also works for some proxy iterator types. In the case of `vector<bool>`, the dereference operator returns an object that is implicitly convertible to `bool`, which can bind to `const bool&`, so `vector<bool>`'s iterators are `Readable`.

But once again we are caught out by move-only types. A `zip` view that zips together a `vector<unique_ptr<int>>` and a `vector<int>` has the following associated types:

Associated type	Value
`ValueType<I>`	`pair<unique_ptr<int>, int>`
`decltype(*i)`	`pair<unique_ptr<int>&, int&>`

To model `Readable`, the expression "`const ValueType<I>& tmp = *i`" must be valid. But trying to initialize a `const pair<unique_ptr<int>, int>&` with a `pair<unique_ptr<int>&, int&>` will fail. It ultimately tries to copy from an lvalue `unique_ptr`. So we see that the `zip` view's iterators are not even `Readable` when one of the element types is move-only. That's unacceptable.

Although the Palo Alto report lifts the restriction that `*i` must be an lvalue expression, we can see from the `Readable` concept that proxy reference types are still not adequately supported.

3.1.3 Constraining higher-order algorithms

The Palo Alto report shows the constrained signature of the `for_each` algorithm as follows:

```
template<InputIterator I, Semiregular F>
  requires Function<F, ValueType<I>>
F for_each(I first, I last, F f);
```

Consider calling code

```
// As before, vi and vs are vectors
auto z = view::zip( vi, vs );
// Let Ref be the zip iterator's reference type:
using Ref = decltype(*z.begin());
// Use for_each to increment all the ints:
for_each( z.begin(), z.end(), [](Ref r) {
    ++r.first;
});
```

Without the constraint, this code compiles. With it, it doesn't. The constraint `Function<F, ValueType<I>>` checks to see if the lambda is callable with `pair<int,string>`. The lambda accepts `pair<int&,string&>`. There is no conversion that makes the call succeed.

Changing the lambda to accept either a "`pair<int,string> [const &]`" or a "`pair<int const &, string const &> [const &]`" would make the check succeed, but the body of the lambda would fail to compile or have the wrong semantics. The addition of the constraint has broken valid code, and there is no way to fix it (short of using a generic lambda).

3.2 Problems with the Cross-type Concepts

The purpose of the `Common` concept in the Palo Alto report is to make cross-type concepts semantically meaningful; it requires that the values of different types, `T` and `U`, can be projected into a shared domain where the operation(s) in question can be performed with identical semantics. Concepts like `EqualityComparable`, `TotallyOrdered`, and `Relation` use `Common` to enforce the semantic coherence that is needed for equational reasoning, even when argument types differ.

However, the syntactic requirements of `Common` cause these concepts to be overconstrained. The "common" type cannot be any random type with no relation to the other two; rather, objects of the original two types must be explicitly convertible to the common type. The somewhat non-intuitive result of this is that `EqualityComparable<T,T>` can be false even when `EqualityComparable<T>` is true. The former has an extra `Common<T,T>` constraint, which is false for non-movable types: there is no such permissible explicit "conversion" from `T` to `T` when `T` is non-movable.

Although not strictly a problem with proxy iterators, the issues with the foundational concepts effect all the parts of the standard library built upon them and so must be addressed. The design described below offers a simple solution to an otherwise thorny problem.

4 Proposed Design

The design suggested here makes heavier use of an existing API, `iter_swap(I,I)`, promoting it to the status of customization point, thereby giving proxy iterators a way to control how elements are swapped. In addition, it suggests a new customization point: `iter_move(I)`, which can be used for moving an element at a certain position out of sequence, leaving a "hole". The return type of `iter_move` is the iterator's *rvalue reference*, a new associated type. The `IndirectlySwappable` and `IndirectlyMovable` concepts are re-expressed in terms of `iter_swap` and `iter_move`, respectively.

The relationships between an iterator's associated types, currently expressed in terms of convertability, are re-expressed in terms of a shared *common reference* type. A *common reference* is much like the familiar `common_type` trait, except that instead of throwing away top-level cv and ref qualifiers, they are preserved. Informally, the common reference of two reference types is the *minimally-qualified* reference type to which both types can bind. Like `common_type`, the new `common_reference` trait can be specialized.

4.1 Impact on the Standard

4.1.1 Overview, for implementers

The algorithms must be specified to use `iter_swap` and `iter_move` when swapping and moving elements. The concepts must be respecified in terms of the new customization points, and a new type trait, `common_reference`, must be specified and implemented. The known shortcomings of `common_type` (e.g., difficulty of specialization) must be addressed. (The formulation of `common_type` given in this paper fixes all known issues.) Care must be taken in the algorithm implementations to hew to the valid expressions for the iterator concepts. The algorithm constraints must be respecified to accommodate proxy iterators.

4.1.2 Overview, for users

For user code, the changes are minimal. Little to no conforming code that works today will stop working after adoping this resolution. The changes to `common_type` are potentially breaking, but only for conversion sequences that are sensitive to cv qualification and value category, and the committee has shown no reluctance to make similar changes to `common_type` before. The addition of `common_reference` gives recourse to users who care about it.

When adapting generic code to work with proxy iterators, calls to `swap` and `move` should be replaced with `iter_swap` and `iter_move`, and for calls to higher-order algorithms, generic lambdas are the preferred solution. When that's not possible, functions can be changed to take arguments by the iterator's *common reference* type, which is the result of applying the `common_reference` trait to `ReferenceType<I>` and `ValueType<I>&`. (An `iter_common_reference_t<I>` type alias is suggested to make this simpler.)

4.1.3 CommonReference and CommonType

The suggested `common_reference` type trait and the `CommonReference` concept that uses it, which is used to express the constraints between an iterator's associated types, takes two (possibly cv- and ref-qualified) types and finds a common type (also possibly qualified) to which they can both be converted *or bound*. When passed two reference types, `common_reference` tries to find another reference type to which both references can bind. (`common_reference` may return a non-reference type if no such reference type is found.) If common references exist between an iterator's associated types, then generic code knows how to manipulate values read from the iterator, and the iterator "makes sense".

Like `common_type`, `common_reference` may also be specialized on user-defined types, and this is the hook that is needed to make proxy references work in a generic context. As a purely practical matter, specializing such a template presents some issues. Would a user need to specialize `common_reference` for every permutation of cv- and ref-qualifiers, for both the left and right arguments? Obviously, such an interface would be broken. The issue is that there is no way in C++ to partially specialize on type *qualifiers*.

Rather, `common_reference` is implemented in terms of another template: `basic_common_reference`. The interface to `basic_common_reference` is given below:

```
template <class T, class U,
  template <class> class TQual,
  template <class> class UQual>
struct basic_common_reference;
```

An instantiation of `common_reference<T cv &, U cv &>` defers to `basic_common_reference<T, U, tqual, uqual>`, where `tqual` is a unary alias template such that `tqual<T>` is `T cv &`. Basically, the template template parameters encode the type qualifiers that got stripped from the first two arguments. That permits users to effectively partially specialize `basic_common_reference` – and hence `common_reference` – on type qualifiers.

For instance, here is the partial specialization that find the common "reference" of two `tuple`s of references – which is a proxy reference.

```
template <class...Ts, class...Us,
  template <class> class TQual,
  template <class> class UQual>
  requires sizeof...(Ts) == sizeof...(Us) &&
    (CommonReference<TQual<Ts>, UQual<Us>>() &&...)
struct basic_common_reference<tuple<Ts...>, tuple<Us...>, TQual, UQual> {
  using type = tuple<
    common_reference_t<TQual<Ts>, UQual<Us>>...>;
};
```

With this specialization, the common reference between the types `tuple<int,double>&` and `tuple<const int&,double&>` is computed as `tuple<const int&,double&>`. (The fact that there is currently no conversion from an lvalue of type `tuple<int,double>` to `tuple<const int&,double&>` means that these two types do not model `CommonReference`. Arguably, such a conversion should exist.)

A reference implementation of `common_type` and `common_reference` can be found in Appendix 1.

4.1.3.1 CommonReference and the Cross-Type Concepts

The `CommonReference` concept also eliminates the problems with the cross-type concepts as described in the section "Problems with the Cross-type Concepts". By using the `CommonReference` concept instead of `Common` in concepts like `EqualityComparable` and `TotallyOrdered`, these concepts are no longer overconstrained since a const lvalue of type "`T`" can bind to the common reference type "`const T&`", regardless of whether `T` is movable or not. `CommonReference`, like `Common`, ensures that there is a shared domain in which the operation(s) in question are semantically meaningful, so equational reasoning is preserved.

4.1.4 Permutable: `iter_swap` and `iter_move`

Today, `iter_swap` is a useless vestige. By expanding its role, we can press it into service to solve the proxy iterator problem, at least in part. The primary `std::swap` and `std::iter_swap` functions get constrained as follows:

```
// swap is defined in <utility>
Movable{T}
void swap(T &t, T &u) noexcept(/*...*/) {
  T tmp = move(t);
  t = move(u);
  u = move(tmp);
}

// Define iter_swap in terms of swap if that's possible
template <Readable R1, Readable R2>
  // Swappable concept defined in new <concepts> header
  requires Swappable<ReferenceType<R1>, ReferenceType<R2>>()
void iter_swap(R1 r1, R2 r2) noexcept(noexcept(swap(*r1, *r2))) {
  swap(*r1, *r2);
}
```

By making `iter_swap` a customization point and requiring all algorithms to use it instead of `swap`, we make it possible for proxy iterators to customize how elements are swapped.

Code that currently uses "`using std::swap; swap(*i1, *i2);`" can be trivially upgraded to this new formulation by doing "`using std::iter_swap; iter_swap(i1, i2)`" instead.

In addition, this paper recommends adding a new customization point: `iter_move`. This is for use by those permuting algorithms that must move elements out of sequence temporarily. `iter_move` is defined essentially as follows:

```
template <class R>
using __iter_move_t =
  conditional_t<
    is_reference_v<ReferenceType<R>>,
    remove_reference_t<ReferenceType<R>> &&,
    decay_t<ReferenceType<R>>;

template <class R>
__iter_move_t<R> iter_move(R r)
  noexcept(noexcept(__iter_move_t<R>(std::move(*r)))) {
  return std::move(*r);
}
```

Code that currently looks like this:

```
ValueType<I> tmp = std::move(*it);
// ...
*it = std::move(tmp);
```

can be upgraded to use `iter_move` as follows:

```
using std::iter_move;
ValueType<I> tmp = iter_move(it);
// ...
*it = std::move(tmp);
```

With `iter_move`, the `Readable` concept picks up an additional associated type: the return type of `iter_move`, which we call `RvalueReferenceType`.

```
template <class R>
using RvalueReferenceType = decltype(iter_move(declval<R>()));
```

This type gets used in the definition of the new iterator concepts described below.

With the existence of `iter_move`, it makes it possible to implement `iter_swap` in terms of `iter_move`, just as the default `swap` is implement in terms of `move`. But to take advantage of all the existing overloads of `swap`, we only want to do that for types that are not already swappable.

```
template <Readable R1, Readable R2>
  requires !Swappable<ReferenceType<R1>, ReferenceType<R2>> &&
    IndirectlyMovable<R1, R2> && IndirectlyMovable<R2, R1>
void iter_swap(R1 r1, R2 r2)
  noexcept(is_nothrow_indirectly_movable_v<R1, R2> &&
           is_nothrow_indirectly_movable_v<R2, R1>) {
  ValueType<R1> tmp = iter_move(r1);
  *r1 = iter_move(r2);
  *r2 = std::move(tmp);
}
```

See below for the updated `IndirectlyMovable` concept.

4.1.5 Iterator Concepts

Rather than requiring that an iterator's `ReferenceType` be convertible to `const ValueType<I>&`– which is overconstraining for proxied sequences – we require that there is a shared reference-like type to which both references and values can bind. The new `RvalueReferenceType` associated type needs a similar constraint.

Only the syntactic requirements are given here. The semantic requirements are described in the Technical Specifications section.

4.1.5.1 Concept Readable

Below is the suggested new formulation for the `Readable` concept:

```
template <class I>
concept bool Readable() {
  return Movable<I>() && DefaultConstructible<I>() &&
    requires (const I& i) {
      // Associated types
      typename ValueType<I>;
      typename ReferenceType<I>;
      typename RvalueReferenceType<I>;

      // Valid expressions
      { *i } -> Same<ReferenceType<I>>;
      { iter_move(i) } -> Same<RvalueReferenceType<I>>;
    } &&
    // Relationships between associated types
    CommonReference<ReferenceType<I>, ValueType<I>&>() &&
    CommonReference<ReferenceType<I>, RvalueReferenceType<I>>() &&
    CommonReference<RvalueReferenceType<I>, const ValueType<I>&>() &&
    // Extra sanity checks (not strictly needed)
    Same<
      CommonReferenceType<ReferenceType<I>, ValueType<I>>,
      ValueType<I>>() &&
    Same<
      CommonReferenceType<RvalueReferenceType<I>, ValueType<I>>,
      ValueType<I>>();
}

// A generally useful dependent type
template <Readable I>
using iter_common_reference_t =
  common_reference_t<ReferenceType<I>, ValueType<I>&>;
```

4.1.5.2 Concepts IndirectlyMovable and IndirectlyCopyable

Often we want to move elements indirectly, from one type that is readable to another that is writable. `IndirectlyMovable` groups the necessary requirements. We can derive those requirements by looking at the implementation of `iter_swap` above that uses

`iter_move`. They are:

1. `ValueType<In> value = iter_move(in)`
2. `value = iter_move(in) // by extension`
3. `*out = iter_move(in)`
4. `*out = std::move(value)`

We can formalize this as follows:

```
template <class In, class Out>
concept bool IndirectlyMovable() {
  return Readable<In>() && Movable<ValueType<In>>() &&
    Constructible<ValueType<In>, RvalueReferenceType<In>>() &&
    Assignable<ValueType<I>&, RvalueReferenceType<In>>() &&
    MoveWritable<Out, RvalueReferenceType<In>>() &&
    MoveWritable<Out, ValueType<I>>();
}
```

Although more strict than the Palo Alto formulation, which only requires `*out = move(*in)`, this concept gives algorithm implementors greater license for storing intermediates when moving elements indirectly, a capability required by many of the permuting algorithms.

The `IndirectlyCopyable` concept is defined similarly:

```
template <class In, class Out>
concept bool IndirectlyCopyable() {
  return IndirectlyMovable<In, Out>() &&
    Copyable<ValueType<In>>() &&
    Constructible<ValueType<In>, ReferenceType<In>>() &&
    Assignable<ValueType<I>&, ReferenceType<In>>() &&
    Writable<Out, ReferenceType<In>>() &&
    Writable<Out, ValueType<I>>();
}
```

4.1.5.3 Concept IndirectlySwappable

With overloads of `iter_swap` that work for `Swappable` types and `IndirectlyMovable` types, the `IndirectlySwappable` concept is trivially implemented in terms of `iter_swap`, with extra checks to test for symmetry:

```
template <class I1, class I2>
concept bool IndirectlySwappable() {
  return Readable<I1>() && Readable<I2>() &&
    requires (I1 i1, I2 i2) {
      iter_swap(i1, i2);
      iter_swap(i2, i1);
      iter_swap(i1, i1);
      iter_swap(i2, i2);
    };
}
```

4.1.6 Algorithm constraints: IndirectCallable

Further problems with proxy iterators arise while trying to constrain algorithms that accept callback functions from users: predicates, relations, and projections. Below, for example, is part of the implementation of `unique_copy` from the SGI STL[7].

```
_Tp value = *first;
*result = value;
while (++first != last)
  if (!binary_pred(value, *first)) {
    value = *first;
    *++result = value;
  }
```

The expression "`binary_pred(value, *first)`" is invoking `binary_pred` with an lvalue of the iterator's value type and its reference type. If `first` is a `vector<bool>` iterator, that means `binary_pred` must be callable with `bool&` and `vector<bool>::reference`. All over the STL, predicates are called with every permutation of `ValueType<I>&` and `ReferenceType<I>`.

The Palo Alto report uses the simple `Predicate<F, ValueType<I>, ValueType<I>>` constraint on such higher-order algorithms. When

an iterator's `operator*` returns an lvalue reference or a non-proxy rvalue, this simple formulation is adequate. The predicate `F` can simply take its arguments by "`const ValueType<I>&`", and everything works.

With proxy iterators, the story is more complicated. As described in the section Constraining higher-order algorithms, the simple constraint formulation of the Palo Alto report either rejects valid uses, forces the user to write inefficient code, or leads to compile errors.

Since the algorithm may choose to call users' functions with every permutation of value type and reference type arguments, the requirements must state that they are *all* required. Below is the list of constraints that must replace a constraint such as `Predicate<F, ValueType<I>, ValueType<I>>`:

- `Predicate<F, ValueType<I>, ValueType<I>>`
- `Predicate<F, ValueType<I>, ReferenceType<I>>`
- `Predicate<F, ReferenceType<I>, ValueType<I>>`
- `Predicate<F, ReferenceType<I>, ReferenceType<I>>`

There is no need to require that the predicate is callable with the iterator's rvalue reference type. The result of `iter_move` in an algorithm is always used to initialize a local variable of the iterator's value type. In addition, we can add one more requirement to give the algorithms the added flexibility of using monomorphic functions internal to their implementation:

- `Predicate<F, iter_common_reference_t<I>, iter_common_reference_t<I>>`

Rather than require that this unwieldy list appear in the signature of every algorithm, we can bundle them up into the `IndirectPredicate` concept, shown below:

```
template <class F, class I1, class I2>
concept bool IndirectPredicate() {
  return Readable<I1>() && Readable<I2>() &&
    Predicate<F, ValueType<I1>, ValueType<I2>>() &&
    Predicate<F, ValueType<I1>, ReferenceType<I2>>() &&
    Predicate<F, ReferenceType<I1>, ValueType<I2>>() &&
    Predicate<F, ReferenceType<I1>, ReferenceType<I2>>() &&
    Predicate<F, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}
```

The algorithm's constraints in the latest Ranges TS draft are already expressed in terms of a simpler set of `Indirect` callable concepts, so this change would mostly be localized to the concept definitions.

From the point of view of the users who must author predicates that satisfy these extra constraints, no changes are needed for any iterator that is valid today; the added constraints are satisfied automatically for non-proxy iterators. When authoring a predicate to be used in conjunction with proxy iterators, the simplest solution is to use a polymorphic lambda for the predicate. For instance:

```
// Polymorphic lambdas will work with proxy iterators:
sort(first, last, [](auto&& x, auto&& y) {return x < y;});
```

If using a polymorphic lambda is undesirable, an alternate solution is to use the iterator's common reference type:

```
// Use the iterator's common reference type to define a monomorphic relation:
using R = iter_common_reference_t<I>;
sort(first, last, [](R&& x, R&& y) {return x < y;});
```

5 Alternate Designs

5.1 Make `move` a customization point

Rather than adding a new customization point (`iter_move`) we could press `std::move` into service as a partial solution to the proxy iterator problem. The idea would be to make expressions like `std::move(*it)` do the right thing for iterators like the `zip` iterator described above. That would involve making `move` a customization point and letting it return something other than an rvalue reference type, so that proxy lvalues can be turned into proxy rvalues. (This solution would still require `common_reference` to solve the other problems described above.)

At first blush, this solution makes a kind of sense: `swap` is a customization point, so why isn't `move`? That logic overlooks the fact that users already have a way to specify how types are moved: with the move constructor and move assignment operator.

Rvalue references and move semantics are complicated enough without adding yet another wrinkle, and overloads of `move` that return something other than `T&&` qualify as a wrinkle; `move` is widely used, and there's no telling how much code could break if `move` returned something other than a `T&&`.

It's also a breaking change since `move` is often called qualified, as `std::move`. That would not find any overloads in other namespaces unless the approach described in N4381[5] were adopted. Turning `move` into a namespace-scoped function object (the customization point design suggested by `N4381`) comes with its own risks, as described in that paper.

Making `move` the customization point instead of `iter_move` reduces design flexibility for authors of proxy iterator types since argument-dependent dispatch happens on the type of the proxy *reference* instead of the iterator. Consider the `zip` iterator described above, whose reference type is a prvalue `pair` of lvalue references. To make this iterator work using `move` as the customization point would require overloading `move` on `pair`. That would be a breaking change since `move` of `pair` already has a meaning. Rather, the `zip` iterator's reference type would have to be some special `proxy_pair` type just so that `move` could be overloaded for it. That's undesirable.

A correlary of the above point involves proxy-like types like `reference_wrapper`. Given an lvalue `reference_wrapper` named "`x`", it's unclear what `move(x)` should do. Should it return an rvalue reference to "`x`", or should it return a temporary object that wraps an rvalue reference to "`x.get()`"? With `move` as a customization point, there is often not enough context to say with certainty what behavior to assign to `move` for a type that stores references.

For all these reasons, this paper prefers to add a new, dedicated API – `iter_move` – whose use is unambiguous.

5.2 New iterator concepts

In N1640[1], Abrahams et.al. describe a decomposition of the standard iterator concept hierarchy into access concepts: `Readable`, `Writable`, `Swappable`, and `Lvalue`; and traversal concepts: `SinglePass`, `Forward`, `Bidirectional`, and `RandomAccess`. Like the design suggested in this paper, the `Swappable` concept from N1640 is specified in terms of `iter_swap`. Since N1640 was written before move semantics, it does not have anything like `iter_move`, but it's reasonable to assume that it would have invented something similar.

Like the Palo Alto report, the `Readable` concept from N1640 requires a convertibility constraint between an iterator's reference and value associated types. As a result, N1640 does not adequately address the proxy reference problem as presented in this paper. In particular, it is incapable of correctly expressing the relationship between a move-only value type and its proxy reference type. Also, the somewhat complicated iterator tag composition suggested by N1640 is not necessary in a world with concept-based overloading.

In other respects, N1640 agrees with the STL design suggested by the Palo Alto report and the Ranges TS, which also has concepts for `Readable` and `Writable`. In the Palo Alto design, these "access" concepts are not purely orthogonal to the "traversal" concepts of `InputIterator`, `ForwardIterator`, however, since the latter are not pure traversal concepts; rather, these iterators are all `Readable`. The standard algorithms have little need for writable-but-not-readable random access iterators, for instance, so a purely orthogonal design does not accurately capture the requirements clusters that appear in the algorithm constraints. The binary concepts `IndirectlyMovable<I,O>`, `IndirectlyCopyable<I,O>`, and `IndirectlySwappable<I1,I2>` from the Palo Alto report do a better job of grouping common requirements and reducing verbosity in the algorithm constraints.

5.3 Cursor/Property Map

N1873[3], the "Cursor/Property Map Abstraction", suggests a radical solution to the proxy iterator problem, among others: break iterators into two distint entities – a cursor that denotes position and a property map that facilitates element access. A so-called property map (`pm`) is a polymorphic function that is used together with a cursor (`c`) to read an element (`pm(*c)`) and with a cursor and value (`v`) to write an element (`pm(*c, v)`). This alternate syntax for element access obviates the need for proxy objects entirely, so the proxy iterator problem simply disappears.

The problems with this approach are mostly practical: the model is more complicated and the migration story is poor. No longer is a single object sufficient for both traversal and access. Three arguments are needed to denote a range: a begin cursor, an end cursor, and a property map. Generic code must be updated to account for the new syntax and for the extra property map argument. In other words, this solution is more invasive than the one this document presents.

5.4 Language support

In private exchange, Sean Parent suggested a more radical fix for the proxy reference problem: change the language. With his suggestion, it would be possible to specify that a type is a proxy reference with a syntax such as:

```
struct bool_reference : bool& {
```

```
    // ...
}
```

Notice the “inheritance” from `bool&`. When doing template type deduction, a `bool_reference` can bind to a `T&`, with `T` deduced to `bool`. This solution has not been explored in depth. It is unclear how to control which operations are to be performed on the proxy itself and which on the object being proxied, or under which circumstances, if any, that is desirable. The impact of changing template type deduction and possibly overload resolution to natively support proxy references is unknown.

6 Technical Specifications

This section is written as a set of diffs against N4382, “C++ Extensions for Ranges” and N4141 (C++14), except where otherwise noted.

6.0.1 Chapter 19: Concepts

To [19.2] Core Language Concepts, add the following:

> **19.2.*X* Concept CommonReference [concepts.lib.corelang.commonref]**
>
> 1. If `T` and `U` can both be explicitly converted or bound to a third type, `C`, then `T` and `U` share a *common reference type*, `C`. [*Note:* `C` could be the same as `T`, or `U`, or it could be a different type. `C` may be a reference type. `C` may not be unique. –*end note*] Informally, two types `T` and `U` model the `CommonReference` concept when the type alias `CommonReferenceType<T, U>` is well-formed and names a common reference type of `T` and `U`.
>
> ```
> template <class T, class U>
> using CommonReferenceType = std::common_reference_t<T, U>;
>
> template <class T, class U>
> concept bool CommonReference() {
>   return
>     requires (T&& t, U&& u) {
>       typename CommonReferenceType<T, U>;
>       typename CommonReferenceType<U, T>;
>       requires Same<CommonReferenceType<T, U>,
>                     CommonReferenceType<U, T>>();
>       CommonReferenceType<T, U>(std::forward<T>(t));
>       CommonReferenceType<T, U>(std::forward<U>(u));
>     };
> }
> ```
>
> 2. Let `C` be `CommonReferenceType<T, U>`. Let `t1` and `t2` be objects of type `T`, and `u1` and `u2` be objects of type `U`. `CommonReference<T, U>()` is satisfied if and only if
>
> (2.1) – `C(t1)` equals `C(t2)` if and only if `t1` equals `t2`.
> (2.2) – `C(u1)` equals `C(u2)` if and only if `u1` equals `u2`.
>
> 3. [Note: Users are free to specialize `common_reference` when at least one parameter is a user-defined type. Those specializations are considered by the `CommonReference` concept. –end note]

Change 19.2.5 Concept Common to the following:

```
template <class T, class U>
using CommonType = std::common_type_t<T, U>;

template <class T, class U>
concept bool Common() {
  return CommonReference<const T&, const U&>() &&
    requires (T&& t, U&& u) {
      typename CommonType<T, U>;
      typename CommonType<U, T>;
      requires Same<CommonType<T, U>,
                    CommonType<U, T>>();
      CommonType<T, U>(std::forward<T>(t));
      CommonType<T, U>(std::forward<U>(u));
      requires CommonReference<CommonType<T, U>&,
                               CommonReferenceType<const T&, const U&>>();
    };
}
```

Change the definitions of the cross-type concepts `Swappable<T,U>` ([concepts.lib.corelang.swappable]), `EqualityComparable<T,U>` ([concepts.lib.compare.equalitycomparable]), `TotallyOrdered<T,U>` ([concepts.lib.compare.totallyordered]), and `Relation<F,T,U>` ([concepts.lib.functions.relation]) to use `CommonReference<const T&, const U&>` instead of `Common<T, U>`.

In addition, `Relation<F,T,U>` requires `Relation<F, CommonReferenceType<const T&, const U&>>` rather than `Relation<F, CommonType<T, U>>`.

6.0.2 Chapter 20: General utilities

To 20.2, add the following to the `<utility>` synopsis (*N.B.*, in namespace `std`):

[*Editorial note:* – Future work: harmonize this with N4426: Adding [nothrow-]swappable traits. *–end note*]

```
// is_nothrow_swappable
template <class R1, class R2>
struct is_nothrow_swappable;

template <class R1, class R2>
constexpr bool is_nothrow_swappable_v = is_nothrow_swappable_t<R1, R2>::value;
```

Add subsection 20.2.6 `is_nothrow_swappable` (*N.B.*, in namespace `std`):

```
template <class T, class U>
struct is_nothrow_swappable : false_type { };
Swappable{T, U}
struct is_nothrow_swappable<T, U> :
  bool_constant<noexcept(swap(declval<T>(), declval<U>()))> { };
```

To 20.10.2, add the following to the `<type_traits>` synopsis (*N.B.*, in namespace `std`):

```
// 20.10.7.6, other transformations:
...
// common_reference
template <class T, class U, template <class> class TQual, template <class> class UQual>
struct basic_common_reference { };
template <class... T> struct common_reference;
...
template <class... T>
  using common_reference_t = typename common_reference<T...>::type;
```

Change Table 57 Other Transformations as follows:

Template	Condition	Comments
`template <class... T>` `struct common_type;`		The member typedef `type` shall be defined or omitted as specified below. If it is omitted, there shall be no member `type`. ~~All types~~Each type in the parameter pack `T` shall be complete or (possibly *cv*) `void`. A program may specialize this trait if at least one template parameter in the specialization is a user-defined type and `sizeof...(T) == 2`. [*Note:* Such specializations are needed only when explicit conversions are desired among the template arguments. *–end note*]

`template <class T, class U,`
`template <class> class TQual,`
`template <class> class UQual>`
`struct basic_common_reference;`
		There shall be no member typedef `type`. A program may specialize this trait if at least one template parameter in the specialization is a user-defined type. [*Note:* – Such specializations may be

	used to influence the result of `common_reference` –*end note*]
`template <class... T>` `struct common_reference;`	The member typedef `type` shall be defined or omitted as specified below. If it is omitted, there shall be no member `type`. Each type in the parameter pack `T` shall be complete or (possibly *cv*) `void`. A program may specialize this trait if at least one template parameter in the specialization is a user-defined type and `sizeof...(T) == 2`. [*Note:* Such specializations are needed to properly handle proxy reference types in generic code. –*end note*]

Delete [meta.trans.other]/p3 and replace it with the following:

3. Let `CREF(A)` be `add_lvalue_reference_t<add_const_t<remove_reference_t<A>>>`. Let `UNCVREF(A)` be `remove_cv_t<remove_reference_t<A>>`. Let `XREF(A)` denote a unary template `T` such that `T<UNCVREF(A)>` denotes the same type as `A`. Let `COPYCV(FROM,TO)` be an alias for type `TO` with the addition of `FROM`'s top-level cv-qualifiers. [*Example:* – `COPYCV(int const, short volatile)` is an alias for `short const volatile`. – *exit example*] Let `COND_RES(X,Y)` be `decltype(declval<bool>()? declval<X>() : declval<Y>())`. Given types `A` and `B`, let `X` be `remove_reference_t<A>`, let `Y` be `remove_reference_t<B>`, and let `COMMON_REF(A,B)` be:

(3.1) – If `A` and `B` are both lvalue reference types, `COMMON_REF(A,B)` is `COND_RES(COPYCV(X,Y) &, COPYCV(Y,X) &)`.
(3.2) – If `A` and `B` are both rvalue reference types, and `COMMON_RES(X&,Y&)` is well formed, and `is_convertible<A,R>::value` and `is_convertible<B,R>::value` are true where `R` is `remove_reference_t<COMMON_RES(X&,Y&)>&&` if `COMMON_RES(X&,Y&)` is a reference type or `COMMON_RES(X&,Y&)` otherwise, then `COMMON_RES(A,B)` is `R`.
(3.3) – If `A` is an rvalue reference and `B` is an lvalue reference and `COMMON_REF(const X&, Y&)` is well formed and `is_convertible<A,R>::value` is true where `R` is `COMMON_REF(const X&, Y&)` then `COMMON_RES(A,B)` is `R`.
(3.4) – If `A` is an lvalue reference and `B` is an rvalue reference, then `COMMON_REF(A,B)` is `COMMON_REF(B,A)`.
(3.5) – Otherwise, `COMMON_REF(A,B)` is `decay_t<COND_RES(CREF(A),CREF(B))>`.

If any of the types computed above are ill-formed, then `COMMON_REF(A,B)` is ill-formed.

4. [*Editorial note:* – The following text in black is taken from the current C++17 draft –*end note*] For the `common_type` trait applied to a parameter pack `T` of types, the member `type` shall be either defined or not present as follows:

(4.1) – If `sizeof...(T)` is zero, there shall be no member `type`.
(4.2) – If `sizeof...(T)` is one, let `T0` denote the sole type in the pack `T`. The member typedef `type` shall denote the same type as `decay_t<T0>`.
(4.3) – If `sizeof...(T)` is two, let `T0` and `T1` denote the two types in the pack `T`, and let `X` and `Y` be `decay_t<T0>` and `decay_t<T1>` respectively. Then

(4.3.1) – If `X` and `T0` denote the same type and `Y` and `T1` denote the same type, then

(4.3.1.1) – If `COMMON_REF(T0,T1)` denotes a valid type, then the member typedef `type` denotes that type.
(4.3.1.2) – Otherwise, there shall be no member `type`.

(4.3.2) – Otherwise, if `common_type_t<X, Y>` denotes a valid type, then the member typedef `type` denotes that type.
(4.3.3) – Otherwise, there shall be no member `type`.

(4.4) – If `sizeof...(T)` is greater than ~~one~~two, let `T1`, `T2`, and `R`, respectively, denote the first, second, and (pack of) remaining types comprising `T`. ~~[*Note:* `sizeof...(R)` may be zero. –*end note*]~~ Let `C` ~~denote the type, if any, of an unevaluated conditional expression (5.16) whose first operand is an arbitrary value of type bool, whose second operand is an xvalue of type T1, and whose third operand is an xvalue of type T2.~~be the type `common_type_t<T1,T2>`. Then

(4.4.1) – If there is such a type `C`, the member typedef `type` shall denote the same type, if any, as `common_type_t<C,R...>`.
(4.4.2) – Otherwise, there shall be no member `type`.

5. For the `common_reference` trait applied to a parameter pack `T` of types, the member `type` shall be either defined or not present as follows:

(5.1) – If `sizeof...(T)` is zero, there shall be no member `type`.
(5.2) – If `sizeof...(T)` is one, let `T0` denote the sole type in the pack `T`. The member typedef `type` shall denote the same type as `T0`.
(5.3) – If `sizeof...(T)` is two, let `T0` and `T1` denote the two types in the pack `T`. Then

(5.3.1) – If `COMMON_REF(T0,T1)` denotes a valid reference type then the member typedef `type` denotes that type.
(5.3.2) – Otherwise, if `basic_common_reference_t<UNCVREF(T0),UNCVREF(T1),XREF(T0),XREF(T1)>` denotes a valid type, then the member typedef `type` denotes that type.
(5.3.3) – Otherwise, if `common_type_t<T0,T1>` denotes a valid type, then the member typedef `type` denotes that type.
(5.3.4) – Otherwise, there shall be no member `type`.

(5.4) – If `sizeof...(T)` is greater than two, let `T1`, `T2`, and `R`, respectively, denote the first, second, and (pack of) remaining types comprising `T`. Let `C` be the type `common_reference_t<T1,T2>`. Then

(5.4.1) – If there is such a type `C`, the member typedef `type` shall denote the same type, if any, as `common_reference_t<C,R...>`.
(5.4.2) – Otherwise, there shall be no member `type`.

6.0.3 Chapter 24. Iterators

Change concept `Readable` ([readable.iterators]) as follows:

```
template <class I>
concept bool Readable() {
  return Movable<I>() && DefaultConstructible<I>() &&
    requires (const I& i) {
      typename ValueType<I>;
      typename ReferenceType<I>;
      typename RvalueReferenceType<I>;
      { *i } -> Same<ReferenceType<I>>;
      { iter_move(i) } -> Same<RvalueReferenceType<I>>;
    } &&
    // Relationships between associated types
    CommonReference<ReferenceType<I>, ValueType<I>&>() &&
    CommonReference<ReferenceType<I>, RvalueReferenceType<I>>() &&
    CommonReference<RvalueReferenceType<I>, const ValueType<I>&>() &&
    Same<
      CommonReferenceType<ReferenceType<I>, ValueType<I>>,
      ValueType<I>>() &&
    Same<
      CommonReferenceType<RvalueReferenceType<I>, ValueType<I>>,
      ValueType<I>>();
}
```

Add a new paragraph (2) to the description of `Readable`:

2. Overload resolution ([over.match]) on the expression `iter_move(i)` selects a unary non-member function "`iter_move`" from a candidate set that includes the `iter_move` function found in `<experimental/ranges_v1/iterator>` ([iterator.synopsis]) and the lookup set produced by argument-dependent lookup ([basic.lookup.argdep]).

Change concept `IndirectlyMovable` ([indirectlymovable.iterators]) to be as follows:

```
template <class In, class Out>
concept bool IndirectlyMovable() {
  return Readable<In>() && Movable<ValueType<In>>() &&
```

```
    Constructible<ValueType<In>, RvalueReferenceType<In>>() &&
    Assignable<ValueType<In>&, RvalueReferenceType<In>>() &&
    MoveWritable<Out, RvalueReferenceType<In>>() &&
    MoveWritable<Out, ValueType<In>>();
}
```

Change the description of the `IndirectlyMovable` concept ([indirectlymovable.iterators]), to be:

2. Let `i` be an object of type `In`, let `o` be a dereferenceable object of type `Out`, and let `v` be an object of type `ValueType<In>`. Then `IndirectlyMovable<In,Out>()` is satisfied if and only if
(2.1) – The expression `ValueType<In>(iter_move(i))` has a value that is equal to the value `*i` had before the expression was evaluated.
(2.2) – After the assignment `v = iter_move(i)`, `v` is equal to the value of `*i` before the assignment.
(2.3) – If `Out` is `Readable`, after the assignment `*o = iter_move(i)`, `*o` is equal to the value of `*i` before the assignment.
(2.4) – If `Out` is `Readable`, after the assignment `*o = std::move(v)`, `*o` is equal to the value of `*i` before the assignment.

Change concept `IndirectlyCopyable` ([indirectlycopyable.iterators]) to be as follows:

```
template <class In, class Out>
concept bool IndirectlyCopyable() {
  return IndirectlyMovable<In, Out>() && Copyable<ValueType<In>>() &&
    Constructible<ValueType<In>, ReferenceType<In>>() &&
    Assignable<ValueType<In>&, ReferenceType<In>>() &&
    Writable<Out, ReferenceType<In>>() &&
    Writable<Out, ValueType<In>>();
}
```

Change the description of the `IndirectlyCopyable` concept ([indirectlycopyable.iterators]), to be:

2. Let `i` be an object of type `In`, let `o` be a dereferenceable object of type `Out`, and let `v` be a `const` object of type `ValueType<In>`. Then `IndirectlyCopyable<In,Out>()` is satisfied if and only if
(2.1) – The expression `ValueType<In>(*i)` has a value that is equal to the value of `*i`.
(2.2) – After the assignment `v = *i`, `v` is equal to the value of `*i`.
(2.3) – If `Out` is `Readable`, after the assignment `*o = *i`, `*o` is equal to the value of `*i`.
(2.4) – If `Out` is `Readable`, after the assignment `*o = v`, `*o` is equal to the value of `v`.

Change concept `IndirectlySwappable` ([indirectlyswappable.iterators]) to be as follows:

```
template <class I1, class I2 = I1>
concept bool IndirectlySwappable() {
  return Readable<I1>() && Readable<I2>() &&
    requires (I1 i1, I2 i2) {
      iter_swap(i1, i2);
      iter_swap(i2, i1);
      iter_swap(i1, i1);
      iter_swap(i2, i2);
    };
}
```

Change the description of `IndirectlySwappable`:

1. Overload resolution ([over.match]) on each of the four `iter_swap` expressions selects a binary non-member function

"`iter_swap`" from a candidate set that includes the two `iter_swap` functions found in `<experimental/ranges_v1/iterator>` ([iterator.synopsis]) and the lookup set produced by argument-dependent lookup ([basic.lookup.argdep]).

2. Given an object `i1` of type `I1` and an object `i2` of type `I2`, `IndirectlySwappable<I1,I2>()` is satisfied if after `iter_swap(i1,i2)`, the value of `*i1` is equal to the value of `*i2` before the call, and *vice versa*.

Swap subsections 24.3.3 ([projected.indirectcallables]) and 24.3.4 ([indirectfunct.indirectcallables]), and change the definition of the `Projected` struct ([projected.indirectcallables]) to the following:

```
template <Readable I, IndirectRegularCallable<I> Proj>
struct Projected {
  using value_type = decay_t<ResultType<FunctionType<Proj>, ValueType<I>>>;
  ResultType<FunctionType<Proj>, ReferenceType<I>> operator*() const;
};

template <WeaklyIncrementable I, IndirectRegularCallable<I> Proj>
struct difference_type<Projected<I, Proj>> {
  using type = DifferenceType<I>;
};
```

Change 24.3.4 "Indirect callables" ([indirectfunc.indirectcallables]) as described as follows: Change `IndirectCallable`, `IndirectRegularCallable`, `IndirectCallablePredicate`, `IndirectCallableRelation`, and `IndirectCallableStrictWeakOrder` as follows:

```
template <class F>
concept bool IndirectCallable() {
  return Function<FunctionType<F>>();
}
template <class F, class I>
concept bool IndirectCallable() {
  return Readable<I>() &&
    Function<FunctionType<F>, ValueType<I>>() &&
    Function<FunctionType<F>, ReferenceType<I>>() &&
    Function<FunctionType<F>, iter_common_reference_t<I>>();
}
template <class F, class I1, class I2>
concept bool IndirectCallable() {
  return Readable<I1>() && Readable<I2>() &&
    Function<FunctionType<F>, ValueType<I1>, ValueType<I2>>() &&
    Function<FunctionType<F>, ValueType<I1>, ReferenceType<I2>>() &&
    Function<FunctionType<F>, ReferenceType<I1>, ValueType<I2>>() &&
    Function<FunctionType<F>, ReferenceType<I1>, ReferenceType<I2>>() &&
    Function<FunctionType<F>, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}

template <class F>
concept bool IndirectRegularCallable() {
  return RegularFunction<FunctionType<F>>();
}
template <class F, class I>
concept bool IndirectRegularCallable() {
  return Readable<I>() &&
    RegularFunction<FunctionType<F>, ValueType<I>>() &&
    RegularFunction<FunctionType<F>, ReferenceType<I>>() &&
    RegularFunction<FunctionType<F>, iter_common_reference_t<I>>();
}
template <class F, class I1, class I2>
concept bool IndirectRegularCallable() {
  return Readable<I1>() && Readable<I2>() &&
    RegularFunction<FunctionType<F>, ValueType<I1>, ValueType<I2>>() &&
    RegularFunction<FunctionType<F>, ValueType<I1>, ReferenceType<I2>>() &&
    RegularFunction<FunctionType<F>, ReferenceType<I1>, ValueType<I2>>() &&
    RegularFunction<FunctionType<F>, ReferenceType<I1>, ReferenceType<I2>>() &&
    RegularFunction<FunctionType<F>, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}

template <class F, class I>
concept bool IndirectCallablePredicate() {
  return Readable<I>() &&
    Predicate<FunctionType<F>, ValueType<I>>() &&
```

```
    Predicate<FunctionType<F>, ReferenceType<I>>() &&
    Predicate<FunctionType<F>, iter_common_reference_t<I>>();
}
template <class F, class I1, class I2>
concept bool IndirectCallablePredicate() {
  return Readable<I1>() && Readable<I2>() &&
    Predicate<FunctionType<F>, ValueType<I1>, ValueType<I2>>() &&
    Predicate<FunctionType<F>, ValueType<I1>, ReferenceType<I2>>() &&
    Predicate<FunctionType<F>, ReferenceType<I1>, ValueType<I2>>() &&
    Predicate<FunctionType<F>, ReferenceType<I1>, ReferenceType<I2>>() &&
    Predicate<FunctionType<F>, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}

template <class F, class I1, class I2 = I1>
concept bool IndirectCallableRelation() {
  return Readable<I1>() && Readable<I2>() &&
    Relation<FunctionType<F>, ValueType<I1>, ValueType<I2>>() &&
    Relation<FunctionType<F>, ValueType<I1>, ReferenceType<I2>>() &&
    Relation<FunctionType<F>, ReferenceType<I1>, ValueType<I2>>() &&
    Relation<FunctionType<F>, ReferenceType<I1>, ReferenceType<I2>>() &&
    Relation<FunctionType<F>, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}

template <class F, class I1, class I2 = I1>
concept bool IndirectCallableStrictWeakOrder() {
  return Readable<I1>() && Readable<I2>() &&
    StrictWeakOrder<FunctionType<F>, ValueType<I1>, ValueType<I2>>() &&
    StrictWeakOrder<FunctionType<F>, ValueType<I1>, ReferenceType<I2>>() &&
    StrictWeakOrder<FunctionType<F>, ReferenceType<I1>, ValueType<I2>>() &&
    StrictWeakOrder<FunctionType<F>, ReferenceType<I1>, ReferenceType<I2>>() &&
    StrictWeakOrder<FunctionType<F>, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}
```

Note: These definitions of `IndirectCallable` and `IndirectCallablePredicate` are less general than the ones in N4382 that they replace. The original definitions are variadic but these handle only up to 2 arguments. The Standard Library never requires callback functions to accept more than two arguments, so the reduced expressive power does not impact the Standard Library; however, it may impact user code. The complication is the need to check callability with a cross-product of the parameters' `ValueType` and `ReferenceType`s, which is difficult to express using Concepts Lite and results in an explosion of tests to be performed as the number of parameters increases.

There are several options for preserving the full expressive power of the N4382 concepts should that prove desirable: (1) Require callability testing only with arguments "`ValueType<Is>...`", "`ReferenceType<Is>..`" , and "`iter_common_reference_t<Is>...`", leaving the other combinations as merely documented constraints that are not required to be tested; (2) Actually test the full cross-product of argument types using meta-programming techniques, accepting the compile-time hit when argument lists get large. (The latter has been tested and shown to be feasable.)

Change 24.6 "Header `<experimental/ranges_v1/iterator>` synopsis" ([iterator.synopsis]) by adding the following to namespace `std::experimental::ranges_v1`:

```
// Exposition only
template <class T>
concept bool _Dereferenceable =
  requires (T& t) { {*t} -> auto&&; };

// iter_move (REF)
template <class R>
  requires _Dereferenceable<R>
auto iter_move(R&& r) noexcept(see below) -> see below;

// is_nothrow_indirectly_movable (REF)
template <class R1, class R2>
struct is_nothrow_indirectly_movable;

template <class R1, class R2>
constexpr bool is_nothrow_indirectly_movable_v = is_nothrow_indirectly_movable_t<R1, R2>::value;

template <_Dereferenceable R>
  requires requires (R& r) { { iter_move(r) } -> auto&&; }
using RvalueReferenceType =
  decltype(iter_move(declval<R&>()));

// iter_swap (REF)
template <class R1, class R2,
  Readable _R1 = remove_reference_t<R1>,
```

```
  Readable _R2 = remove_reference_t<R2>>
  requires Swappable<ReferenceType<_R1>, ReferenceType<_R2>>()
void iter_swap(R1&& r1, R2&& r2)
  noexcept(is_nothrow_swappable_v<ReferenceType<_R1>, ReferenceType<_R2>>);

template <class R1, class R2,
  Readable _R1 = std::remove_reference_t<R1>,
  Readable _R2 = std::remove_reference_t<R2>>
  requires !Swappable<ReferenceType<_R1>, ReferenceType<_R2>>()
    && IndirectlyMovable<_R1, _R2>() && IndirectlyMovable<_R2, _R1>()
void iter_swap(R1&& r1, R2&& r2)
  noexcept(is_nothrow_indirectly_movable_v<_R1, _R2> &&
           is_nothrow_indirectly_movable_v<_R2, _R1>);

// is_nothrow_indirectly_swappable (REF)
template <class R1, class R2>
struct is_nothrow_indirectly_swappable;

template <class R1, class R2>
constexpr bool is_nothrow_indirectly_swappable_v = is_nothrow_indirectly_swappable_t<R1, R2>::value;

template <Readable I>
using iter_common_reference_t =
  common_reference_t<ReferenceType<I>, ValueType<I>&>;
```

Before subsubsection “Iterator associated types” ([iterator.assoc]), add a new subsubsection “Iterator utilities” ([iterator.utils]). Under that subsubsection, insert the following:

```
template <class R>
  requires _Dereferenceable<R>
auto iter_move(R&& r) noexcept(see below) -> see below;
```

1. The return type is `Ret` where `Ret` is `remove_reference_t<ReferenceType<R>>&&` if `R` is a reference type; `decay_t<R>`, otherwise.
2. The expression in the `noexcept` is equivalent to:

```
noexcept(Ret(std::move(*r)))
```

3. *Returns:* `std::move(*r)`

[*Editorial note:* – Future work: Rather than defining a new `iter_swap` in namespace `std::experimental::ranges_v1`, it will probably be necessary to constrain the `iter_swap` in namespace `std` much the way the Ranges TS constrains `std::swap`. *–end note*]

```
template <class R1, class R2,
  Readable _R1 = remove_reference_t<R1>,
  Readable _R2 = remove_reference_t<R2>>
  requires Swappable<ReferenceType<_R1>, ReferenceType<_R2>>()
void iter_swap(R1&& r1, R2&& r2)
  noexcept(is_nothrow_swappable_v<ReferenceType<_R1>, ReferenceType<_R2>>);
```

4. *Effects*: `swap(*r1, *r2)`

```
template <class R1, class R2,
  Readable _R1 = std::remove_reference_t<R1>,
  Readable _R2 = std::remove_reference_t<R2>>
  requires !Swappable<ReferenceType<_R1>, ReferenceType<_R2>>()
    && IndirectlyMovable<_R1, _R2>() && IndirectlyMovable<_R2, _R1>()
void iter_swap(R1&& r1, R2&& r2)
  noexcept(is_nothrow_indirectly_movable_v<_R1, _R2> &&
           is_nothrow_indirectly_movable_v<_R2, _R1>);
```

5. *Effects*: Exchanges values referred to by two `Readable` objects.

6. [*Example:* Below is a possible implementation:

```
ValueType<_R1> tmp(iter_move(r1));
```

```
*r1 = iter_move(r2);
*r2 = std::move(tmp);
```

– *end example*]

To [iterator.assoc] (24.7.1), add the following definition of `RvalueReferenceType` by changing this:

> [...] In addition, the type
>
> ```
> ReferenceType<Readable>
> ```
>
> shall be an alias for `decltype(*declval<Readable>())`.

... to this:

> [...] In addition, the alias templates `ReferenceType` and `RvalueReferenceType` shall be defined as follows:
>
> ```
> template <_Dereferenceable R>
> using ReferenceType = decltype(*declval<R&>());
> template <_Dereferenceable R>
>   requires requires (R& r) {
>     { iter_move(r) } -> auto&&;
>   }
> using RvalueReferenceType =
>   decltype(iter_move(declval<R&>()));
> ```
>
> Overload resolution (13.3) on the expression `iter_move(t)` selects a unary non-member function "`iter_move`" from a candidate set that includes the function `iter_move` in `<experimental/ranges_v1/iterator>` (24.6) and the lookup set produced by argument-dependent lookup (3.4.2).

After subsubsection "Iterator operations" ([iterator.operations]), add a new subsubsection "Iterator traits" ([iterator.traits]). Under that subsubsection, include the following:

```
template <class In, class Out>
struct is_nothrow_indirectly_movable : false_type { };
IndirectlyMovable{In, Out}
struct is_nothrow_indirectly_movable<In, Out> :
  bool_constant<
    is_nothrow_constructible<ValueType<In>, RvalueReferenceType<In>>::value &&
    is_nothrow_assignable<ValueType<In> &, RvalueReferenceType<In>>::value &&
    is_nothrow_assignable<ReferenceType<Out>, RvalueReferenceType<In>>::value &&
    is_nothrow_assignable<ReferenceType<Out>, ValueType<In>>::value>
{ };

template <class In, class Out>
struct is_nothrow_indirectly_swappable : false_type { };
IndirectlySwappable{In, Out}
struct is_nothrow_indirectly_swappable<In, Out> :
  bool_constant<
    noexcept(iter_swap(declval<R1>(), declval<R2>())) &&
    noexcept(iter_swap(declval<R2>(), declval<R1>())) &&
    noexcept(iter_swap(declval<R1>(), declval<R1>())) &&
    noexcept(iter_swap(declval<R2>(), declval<R2>()))>
{ };
```

Change 25.1 "Algorithms: General" ([algorithms.general]) as follows:

```
template<InputIterator I, Sentinel<I> S, WeaklyIncrementable O,
    class Proj = identity, IndirectCallableRelation<Projected<I, Proj>> R = equal_to<>>
  requires IndirectlyCopyable<I, O>() && (ForwardIterator<I>() ||
    ForwardIterator<O>() || Copyable<ValueType<I>>())
  tagged_pair<tag::in(I), tag::out(O)>
    unique_copy(I first, S last, O result, R comp = R{}, Proj proj = Proj{});

template<InputRange Rng, WeaklyIncrementable O, class Proj = identity,
    IndirectCallableRelation<Projected<IteratorType<Rng>, Proj>> R = equal_to<>>
  requires IndirectlyCopyable<IteratorType<Rng>, O>() &&
    (ForwardIterator<IteratorType<Rng>>() || ForwardIterator<O>()
```

```
        || Copyable<ValueType<IteratorType<Rng>>>())
    tagged_pair<tag::in(safe_iterator_t<Rng>), tag::out(O)>
      unique_copy(Rng&& rng, O result, R comp = R{}, Proj proj = Proj{});
```

Make the identical change to 25.3.9 “Unique” ([alg.unique]).

7 Acknowledgements

I would like to extend my sincerest gratitude to Sean Parent, Andrew Sutton, and Casey Carter for their help formalizing and vetting many of the ideas presented in this paper and in the Ranges TS.

I would also like to thank Herb Sutter and the Standard C++ Foundation, without whose generous financial support this work would not be possible.

References

[1] Abrahams, D. et al. 2004. N1640: New Iterator Concepts.

[2] CMCSTL2: *https://github.com/CaseyCarter/cmcstl2*. Accessed: 2015-09-09.

[3] Dietmar, K. and Abrahams, D. 2005. N1873: The Cursor/Property Map Abstraction.

[4] Niebler, E. 2015. N4382: Working Draft: C++ Extensions for Ranges.

[5] Niebler, E. 2015. Suggested Design for Customization Points.

[6] Range v3: *http://www.github.com/ericniebler/range-v3*. Accessed: 2014-10-08.

[7] SGI Standard Template Library Programmer’s Guide: *https://www.sgi.com/tech/stl/*. Accessed: 2015-08-12.

[8] Stroustrup, B. and Sutton, A. 2012. N3351: A Concept Design for the STL.

[9] Sutter, H. 1999. When is a container not a container? *C++ Report*. 11, 5 (May 1999).

1 Appendix 1: Reference implementations of `common_type` and `common_reference`

```
#include <utility>
#include <type_traits>

using std::is_same;
using std::decay_t;
using std::declval;

template <class T>
using __t = typename T::type;

template <class T>
constexpr typename __t<T>::value_type __v = __t<T>::value;

template <class T, class... Args>
using __apply = typename T::template apply<Args...>;

template <class T, class U>
struct __compose {
  template <class V>
  using apply = __apply<T, __apply<U, V>>;
};

template <class T>
struct __id { using type = T; };

template <template <class...> class T, class... U>
concept bool _Valid = requires { typename T<U...>; };

template <class U, template <class...> class T, class... V>
```

```
concept bool _Is = _Valid<T, U, V...> && __v<T<U, V...>>;

template <class U, class V>
concept bool _ConvertibleTo = _Is<U, std::is_convertible, V>;

template <template <class...> class T, class... U>
struct __defer { };
_Valid{T, ...U}
struct __defer<T, U...> : __id<T<U...>> { };

template <template <class...> class T>
struct __q {
  template <class... U>
  using apply = __t<__defer<T, U...>>;
};

template <class T>
struct __has_type : std::false_type { };
template <class T> requires _Valid<__t, T>
struct __has_type<T> : std::true_type { };

template <class T, class X = std::remove_reference_t<T>>
using __cref = std::add_lvalue_reference_t<std::add_const_t<X>>;
template <class T>
using __uncvref = std::remove_cv_t<std::remove_reference_t<T>>;

template <class T, class U>
using __cond = decltype(true ? declval<T>() : declval<U>());

template <class From, class To>
struct __copy_cv_ : __id<To> { };
template <class From, class To>
struct __copy_cv_<From const, To> : std::add_const<To> { };
template <class From, class To>
struct __copy_cv_<From volatile, To> : std::add_volatile<To> { };
template <class From, class To>
struct __copy_cv_<From const volatile, To> : std::add_cv<To> { };
template <class From, class To>
using __copy_cv = __t<__copy_cv_<From, To>>;

template <class T, class U>
struct __builtin_common { };
template <class T, class U>
using __builtin_common_t = __t<__builtin_common<T, U>>;
template <class T, class U>
  requires _Valid<__cond, __cref<T>, __cref<U>>
struct __builtin_common<T, U> :
  std::decay<__cond<__cref<T>, __cref<U>>> { };
template <class T, class U, class R = __builtin_common_t<T &, U &>>
using __rref_res = std::conditional_t<__v<std::is_reference<R>>,
  std::remove_reference_t<R> &&, R>;
template <class T, class U>
  requires _Valid<__builtin_common_t, T &, U &>
    && _ConvertibleTo<T &&, __rref_res<T, U>>
    && _ConvertibleTo<U &&, __rref_res<T, U>>
struct __builtin_common<T &&, U &&> : __id<__rref_res<T, U>> { };
template <class T, class U>
using __lref_res = __cond<__copy_cv<T, U> &, __copy_cv<U, T> &>;
template <class T, class U>
struct __builtin_common<T &, U &> : __defer<__lref_res, T, U> { };
template <class T, class U>
  requires _Valid<__builtin_common_t, T &, U const &>
    && _ConvertibleTo<U &&, __builtin_common_t<T &, U const &>>
struct __builtin_common<T &, U &&> :
  __builtin_common<T &, U const &> { };
template <class T, class U>
struct __builtin_common<T &&, U &> : __builtin_common<U &, T &&> { };

// common_type
template <class ...Ts>
struct common_type { };

template <class... T>
using common_type_t = __t<common_type<T...>>;

template <class T>
struct common_type<T> : std::decay<T> { };

template <class T>
```

```
concept bool _Decayed = __v<is_same<decay_t<T>, T>>;

template <class T, class U>
struct __common_type2
  : common_type<decay_t<T>, decay_t<U>> { };

template <_Decayed T, _Decayed U>
struct __common_type2<T, U> : __builtin_common<T, U> { };

template <class T, class U>
struct common_type<T, U> : __common_type2<T, U> { };

template <class T, class U, class V, class... W>
  requires _Valid<common_type_t, T, U>
struct common_type<T, U, V, W...>
  : common_type<common_type_t<T, U>, V, W...> { };

namespace __qual {
  using __rref = __q<std::add_rvalue_reference_t>;
  using __lref = __q<std::add_lvalue_reference_t>;
  template <class>
  struct __xref : __id<__compose<__q<__t>, __q<__id>>> { };
  template <class T>
  struct __xref<T&> : __id<__compose<__lref, __t<__xref<T>>>> { };
  template <class T>
  struct __xref<T&&> : __id<__compose<__rref, __t<__xref<T>>>> { };
  template <class T>
  struct __xref<const T> : __id<__q<std::add_const_t>> { };
  template <class T>
  struct __xref<volatile T> : __id<__q<std::add_volatile_t>> { };
  template <class T>
  struct __xref<const volatile T> : __id<__q<std::add_cv_t>> { };
}

template <class T, class U, template <class> class TQual,
  template <class> class UQual>
struct basic_common_reference { };

template <class T, class U>
using __basic_common_reference =
  basic_common_reference<__uncvref<T>, __uncvref<U>,
    __qual::__xref<T>::type::template apply,
    __qual::__xref<U>::type::template apply>;

// common_reference
template <class... T>
struct common_reference { };

template <class... T>
using common_reference_t = __t<common_reference<T...>>;

template <class T>
struct common_reference<T> : __id<T> { };

template <class T, class U>
struct __common_reference2
  : std::conditional_t<__v<__has_type<__basic_common_reference<T, U>>>,
      __basic_common_reference<T, U>, common_type<T, U>> { };

template <class T, class U>
  requires _Valid<__builtin_common_t, T, U>
    && _Is<__builtin_common_t<T, U>, std::is_reference>
struct __common_reference2<T, U> : __builtin_common<T, U> { };

template <class T, class U>
struct common_reference<T, U> : __common_reference2<T, U> { };

template <class T, class U, class V, class... W>
  requires _Valid<common_reference_t, T, U>
struct common_reference<T, U, V, W...>
  : common_reference<common_reference_t<T, U>, V, W...> { };
```